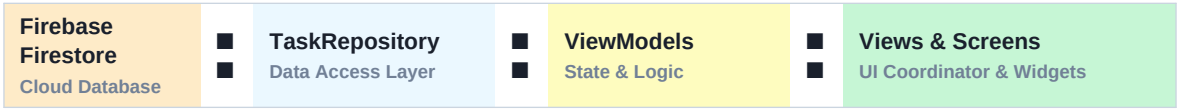
TASK MANAGER APP — Codebase & Architecture Guide

Complete Breakdown of MVVM Structure, Folder Roles, File Connections & Data Flow

Stack: Flutter 3 • Dart • Firebase Firestore & FCM • Provider • GoRouter | **Status:** 100% Tested & Verified

1. The Big Picture: MVVM Architecture

Yeh application **MVVM (Model - View - ViewModel) + Repository Pattern** follow karti hai. Iska basic rule hai ki **UI (Screens/Widgets) bilkul 'Dumb'** rahegi aur sara business logic, state management aur text parsing **ViewModels** me rahega.



Char Mukhya Layers:

- **1. Model Layer:** Data classes jo schema define karti hain (e.g. Task, FloatingSheetType).
- **2. Repository Layer:** Firebase Firestore se data fetch aur save karta hai. Iska UI se koi direct relation nahi hota.
- **3. ViewModel Layer:** State aur Business logic hold karta hai (ChangeNotifier use karke). Tasks filter karna, sorting, mention state sab yahan hota hai.
- **4. View Layer:** User interface (UI). ViewModel ke data ko render karta hai aur user clicks ko ViewModel ke pass bhejta hai.

2. Folder Breakdown (Kaunsa Folder kya karta hai?)

Folder Path	Layer	Description & Main Responsibility
`lib/routes/`	**Navigation**	GoRouter rules. Route / par MapScreen ko ViewModels inject karta hai.
`lib/services/`	**Services**	Firebase Messaging (FCM), Home widget service, aur web shortcut handlers.
`map_screen/models/`	**Model**	Task class, FloatingSheetType enum, aur quick date options.
`map_screen/repositories/`	**Repository**	TaskRepository: Firestore se CRUD operations (watchTasks, addTask, updateTask, deleteTask).
`map_screen/services/`	**Services**	Task assignment push notifications aur Share/PDF print services.
`map_screen/utils/`	**Utils**	task_mention_parser.dart (@, -, !, * shortcuts), date formatter, task constants.
`map_screen/viewmodels/`	**ViewModel**	MapScreenViewModel (tasks list, sort, search, selection) aur AddTaskViewModel.
`map_screen/screens/`	**View Coordinator**	map_screen.dart (Clean coordinator ~267 lines) aur task_detail_page.dart.
`map_screen/screens/views/`	**View Sub-modules**	AppBar, TabViews (8 tabs), AddTaskBottomSheet composer, FloatingMetadataSheet dropdown.
`map_screen/screens/dialogs/`	**Dialogs**	Shortcuts help dialog, zoom slider, notifications modal, delete confirmation.
`map_screen/screens/widgets/`	**Atomic Widgets**	Reusable tiles (MapTaskListItem, SheetOptionButton, metadata badges).

3. File-by-File Catalog (Kaunsi File me kya Code hai?)

Refactoring ke baad har ek file ki single responsibility fix kar di gayi hai:

File Name	Role & Lines	Key Methods & Code Details
`map_screen.dart`	**Coordinator** ~267 lines	Screen skeleton. AppBar, Search bar, TabBar (8 tabs) aur Add Task FAB ko assemble karta hai. ViewModel se data listen karke UI updates trigger karta hai.
`map_screen_viewmodel.dart`	**ViewModel** ~620 lines	Central state: tasks, listenForTasks(), addTask(), toggleTaskDone(), sectionsForWork(), multi-selection state, sort/group/zoom filters.
`add_task_viewmodel.dart`	**ViewModel** ~120 lines	Task creation form state: priority, deadline, assignees, draggable button order (SharedPreferences me save/load), submitTask().
`task_repository.dart`	**Repository** ~140 lines	Firestore calls: watchTasks() (realtime snapshot stream), addTask(), updateTask(), deleteTask(), loadClients, loadAssignees.
`task_mention_parser.dart`	**Utility** ~90 lines	Text shortcuts parsing: parseMention() (@, -, !, *, +, #, ^), matchesSearchQuery() (fuzzy search), removeMentionToken().
`add_task_bottom_sheet.dart`	**View Component** ~700 lines	Bottom sheet composer: Autocomplete mentions popup, metadata button pills bar, keyboard focus management (refocusComposer), task submit button.
`floating_metadata_sheet.dart`	**View Overlay** ~410 lines	Dropdown overlay menu: Dynamic coordinate positioning, search filter in options, custom date picker integration, single/multi-select support.
`map_screen_tab_views.dart`	**View Component** ~315 lines	8 Tab Views: MyDay, MyWork, Assign (openAssigneeOnTap), AllWork, Done, Overdue, Pending, GroupBy. Har task tile ka click aur selection handle karta hai.
`map_screen_app_bar.dart`	**View Component** ~120 lines	Dual mode AppBar: Normal Mode (Notification badge count, Search toggle, More options) aur Multi-select Mode (Delete selected tasks, Selected count, Cancel).
`map_screen_toolbar_menu.dart`	**View Component** ~125 lines	More options popup: Sort by dialog, Group by dialog, View type dialog, Zoom slider dialog, Add to home screen shortcut, Share list, Print PDF.
`assignee_tasks_view.dart`	**View Screen** ~105 lines	Kisi specific assignee ke assigned tasks ko MyWorkView format (Urgent, Today, Tomorrow) me render karta hai.
`task_metadata_badges.dart`	**Widget** ~45 lines	Task tiles ke niche @Assignee, #ClientName, +WorkType ke badges draw karta hai.

4. Data Flow: Kaunsi File Kisse Connect Hai?

Scenario A: App Launch hone par Data Load ka Flow

1. main.dart app initialize karta hai aur AppRouter call hota hai.
2. app_router.dart home route par MapScreenViewModel create karke MapScreen ko provide karta hai.
3. MapScreen ki didChangeDependencies() me viewModel.listenForTasks(authVM) start hota hai.
4. MapScreenViewModel direct TaskRepository.watchTasks() se Firestore snapshot stream start karta hai.
5. Firestore se tasks aate hain -> ViewModel user role check karke filter karta hai (isTaskVisibleToUser).
6. ViewModel notifyListeners() call karta hai -> MapScreenTabViews automatically render ho jati hai.

Scenario B: Naya Task Add hone ka Flow

1. User + button dabata hai -> AddTaskBottomSheet.show(context) bottom sheet open karta hai.
2. User type karta hai 'Meeting @Aman !Today' -> task_mention_parser.dart shortcuts detect karta hai.
3. Metadata values AddTaskViewModel me save hoti hain.

4. User **Send** icon dabata hai -> addVm.submitTask(...) call hota hai.
5. Yeh mapViewModel.addTask(...) ko call karta hai, jo TaskRepository.addTask(task) se Firestore me save karta hai.
6. Saath hi task_assignment_notification_service.dart assignee ko push notification bhej deta hai.

5. 'Ye Hume Kaise Pata Chalega?' (Code Tracing Guide)

Agar aage chalkar aapko codebase me koi nayi cheez trace karni ho ya debugging karni ho, toh yeh 4 golden rules follow karein:

Rule 1: Top Imports Dekho	Kisi bhi file ke top par import ... dekhein. Agar kisi view file me import '../viewmodels/map_screen_viewmodel.dart' hai, toh iska seedha matlab hai ki yeh UI file ViewModel se judi hui hai.
Rule 2: Provider Calls Dhundho	File me Provider.of<ViewModelName>(context) ya context.watch() search karein. Jiss ViewModel ka naam likha hai, UI usi se data le rahi hai aur wahi uski state ka source hai.
Rule 3: F12 / Ctrl+Click (Go to Definition)	VS Code me kisi bhi function ya variable par F12 dabane par editor seedhe us file par le jata hai jahan wo function define hai. Shift + F12 se pata chalta hai ki wo pure project me kahan-kahan use ho raha hai.
Rule 4: AppRouter Check Karein	Nayi screen ka connection samajhne ke liye lib/routes/app_router.dart dekhein. Wahan saaf dikhta hai ki kaunsa Route kis Screen ko open karta hai aur usme kaunse ViewModels pass kiye gaye hain.

6. Interview & Team Presentation Pitch (Cheat Sheet)

Ready-to-Speak Pitch:

*"Humara Task Manager project ek clean **MVVM (Model-View-ViewModel) + Repository Pattern** par architected hai. Data layer me **TaskRepository** Firestore backend se real-time streams handle karta hai. Business logic **MapScreenViewModel** aur **AddTaskViewModel** me encapsulated hai jo state, role-based filtering, aur mention shortcuts (@, -, !, *) manage karte hain. View layer ko humne completely modular rakha hai jahan **MapScreen** sirf ek lightweight coordinator hai (~267 lines), aur saare 8 UI tabs, floating overlays, aur task composers standalone widgets ke roop me clean aur reusable hain."*